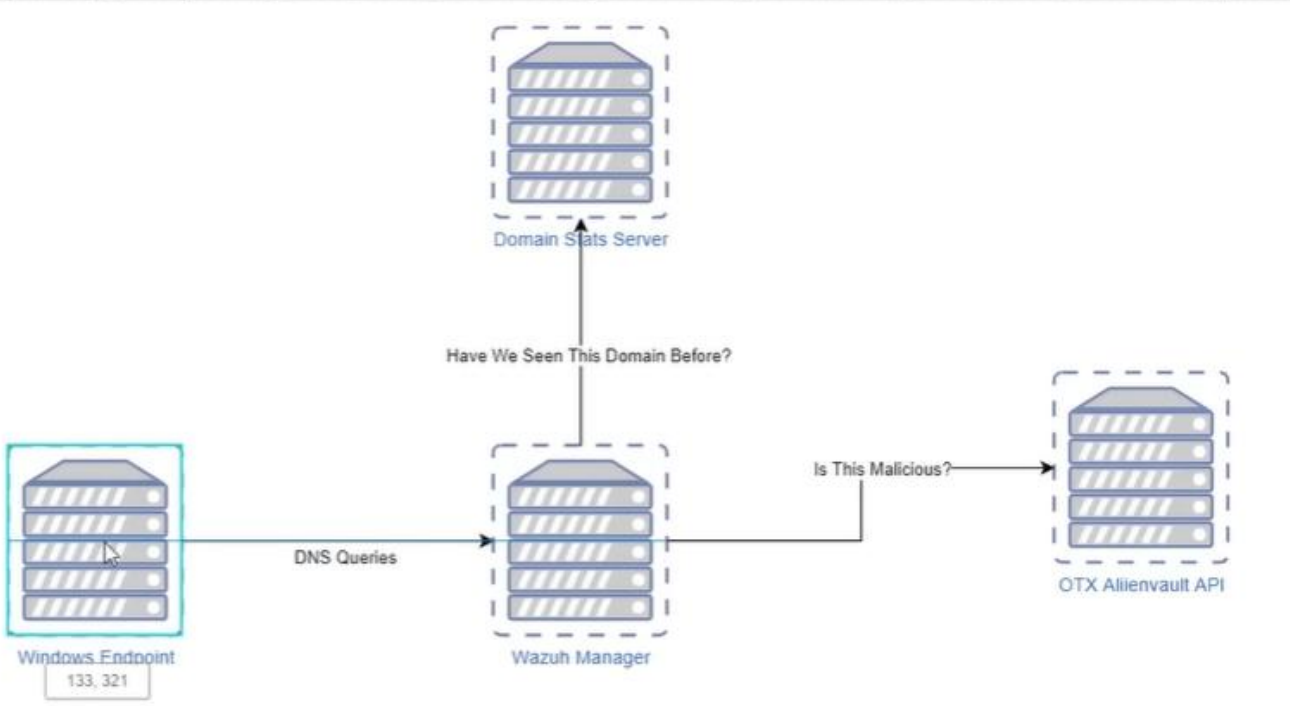
Hunting Malicious DNS Queries from Windows Endpoints using Wazuh, DNS-Stats, and AlienVault OTX

wazuh.



Project Overview

In modern enterprise environments, **DNS is one of the earliest indicators of compromise (IOC)**. Malware, C2 frameworks, phishing kits, and post-exploitation tools almost always rely on DNS before establishing full command-and-control communication.

This project focuses on building a **DNS-based threat hunting and enrichment pipeline** using:

- **Wazuh SIEM**
- **Sysmon Event ID 22 (DNS Query)**
- **DNS-Stats (Domain reputation & frequency analysis)**
- **AlienVault OTX (Threat Intelligence IOC validation)**

The goal is to:

- Detect **new, rare, or suspicious DNS queries** from Windows endpoints
- Enrich them with **historical reputation and frequency data**
- Validate them against **AlienVault OTX threat intelligence**
- Present analysts with **context-rich alerts** mapped to **MITRE ATT&CK**

This creates a **real SOC-style workflow** where alerts are no longer raw logs but actionable intelligence.

Why This Matters (Problem Statement)

Traditional SIEM alerts often fail because:

- DNS queries are noisy
- Not all malicious domains are blocked
- Analysts lack historical and reputation context

This project solves that by:

- Detecting **first-seen domains**
- Identifying **low-frequency / suspicious domains**
- Correlating with **external threat intelligence**
- Enriching alerts before analysts even investigate

What is DNS-Stats?

DNS-Stats is a domain analysis service that provides:

- Domain age
- First-seen timestamps
- Frequency scores
- Reputation signals (RDAP, WHOIS, web history)

This helps answer:

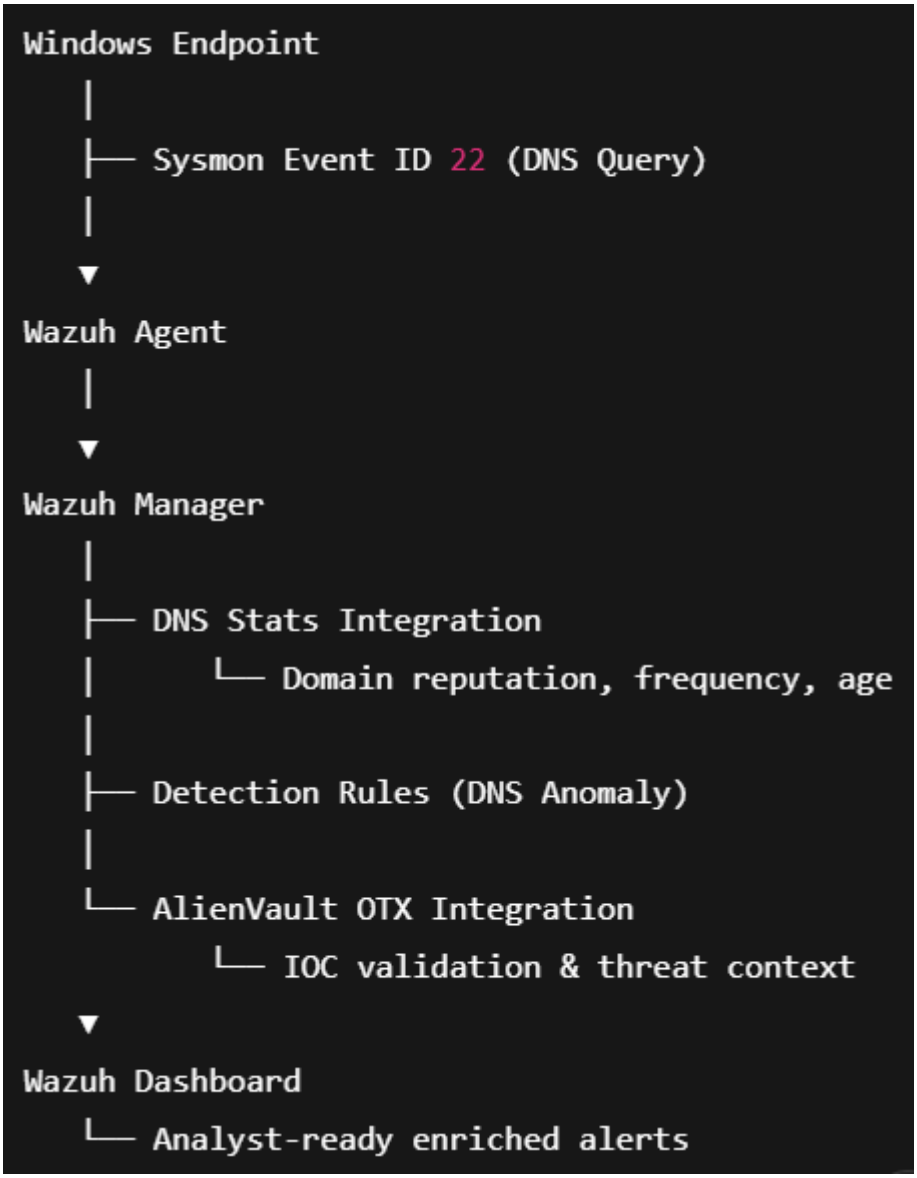
“Has this domain ever been seen before in our environment or globally?”

Tools & Technologies Used

- **Wazuh SIEM**
- **Windows Wazuh Agent**
- **Sysmon**
- **DNS-Stats (Python + Flask + Gunicorn)**
- **AlienVault OTX API**
- **Python**
- **PowerShell**
- **Active Response (Wazuh)**
- **MITRE ATT&CK Framework**

Architecture Overview

Flow of Detection & Enrichment



Step-by-Step Implementation

Step 1: Install DNS-Stats Dependencies

DNS-Stats is a Python-based service.
We install it using pip, which automatically installs dependencies like:

- Flask
- Gunicorn
- Requests
- RDAP libraries

Purpose:
Prepare the Python environment required to run DNS-Stats as an API service.

```
(venv) root@wazuhserver:/home/ubuntu/domain_stats/domain_stats# python3 -m pip install domain-stats
Collecting domain-stats
  Downloading domain_stats-1.0.1-py3-none-any.whl.metadata (19 kB)
Collecting diskcache>=5.1.0 (from domain-stats)
  Downloading diskcache-5.6.3-py3-none-any.whl.metadata (20 kB)
Collecting Flask>=1.1.2 (from domain-stats)
  Downloading flask-3.1.2-py3-none-any.whl.metadata (3.2 kB)
Collecting gunicorn>=20.0.4 (from domain-stats)
  Downloading gunicorn-25.0.1-py3-none-any.whl.metadata (4.7 kB)
Collecting publicsuffixlist>=0.7.6 (from domain-stats)
  Downloading publicsuffixlist-1.0.2.20260204-py2.py3-none-any.whl.metadata (5.9 kB)
Collecting python-dateutil>=2.8.1 (from domain-stats)
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
```

```
Downloading typing_inspection-0.4.2-py3-none-any.whl (14 kB)
Building wheels for collected packages: googlemaps
  Building wheel for googlemaps (pyproject.toml) ... done
  Created wheel for googlemaps: filename=googlemaps-4.10.0-py3-none-any.whl size=40820 sha256=abb9f233dd9e231c3a2f4b332ccb6219ab5b80e6b1b5b2309b8fdfbb0d82d75f
  Stored in directory: /root/.cache/pip/wheels/4c/6a/a7/bbc6f5c200032025ee655deb5e163ce8594fa05e67d973aad6
Successfully built googlemaps
Installing collected packages: urllib3, typing-extensions, six, PyYAML, publicsuffixlist, phonenumbers, packaging, markupsafe, itsdangerous, idna, diskcache, click, chardet,
 certifi, blinker, annotated-types, werkzeug, typing-inspection, requests, python-dateutil, pydantic-core, jinja2, gunicorn, pydantic, munge, googlemaps, Flask, rdap, domain
-stats
Successfully installed Flask-3.1.2 PyYAML-6.0.3 annotated-types-0.7.0 blinker-1.9.0 certifi-2026.1.4 chardet-4.0.0 click-8.3.1 diskcache-5.6.3 domain-stats-1.0.1 googlemaps-
4.10.0 gunicorn-25.0.1 idna-2.10 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 munge-1.3.0 packaging-26.0 phonenumbers-9.0.23 publicsuffixlist-1.0.2.20260204 pydantic-2.1
2.5 pydantic-core-2.41.5 python-dateutil-2.9.0.post0 rdap-1.7.0 requests-2.25.1 six-1.17.0 typing-extensions-4.15.0 typing-inspection-0.4.2 urllib3-1.26.20 werkzeug-3.1.5
```

Step 2: Clone and Install DNS-Stats

The DNS-Stats repository is cloned and installed locally.

Purpose:
This provides the core logic for:

- Domain frequency scoring
- Reputation lookups
- Historical domain analysis

```
(venv) root@wazuhserver:/home/ubuntu/domain_stats/domain_stats# git clone https://github.com/MarkBaggett/domain_stats
cd domain_stats
python3 -m pip install .
```

```
Building wheels for collected packages: domain_stats
  Building wheel for domain_stats (pyproject.toml) ... done
  Created wheel for domain_stats: filename=domain_stats-1.0.1-py3-none-any.whl size=1444436 sha256=05794b29791eb70b74b5eb1509f437c177ced1a2c6835591c999f441da8bfdbd
  Stored in directory: /tmp/pip-ephem-wheel-cache-g7r50_ml/wheels/cd/87/5e/625b762c27085bf16e634ad662417c8240287a60578a055ccb
Successfully built domain_stats
Installing collected packages: domain_stats
  Attempting uninstall: domain_stats
    Found existing installation: domain-stats 1.0.1
    Uninstalling domain-stats-1.0.1:
      Successfully uninstalled domain-stats-1.0.1
Successfully installed domain_stats-1.0.1
```

Step 3: Configure DNS-Stats

DNS-Stats is configured with:

- Listening IP (localhost)
- Port (5730)
- Cache and frequency settings
- Analysis thresholds

Purpose:
Tune how DNS-Stats evaluates domains and where it listens for requests.

```
(venv) root@wazuhserver:/home/ubuntu/domain_stats/domain_stats/domain_stats# domain-stats-settings /opt/domain-stats-data
Set value for ip_address. Default=127.0.0.1 Current=127.0.0.1 (Enter to keep Current):
Set value for local_port. Default=5730 Current=5730 (Enter to keep Current):
Set value for workers. Default=5 Current=5 (Enter to keep Current):
Set value for threads_per_worker. Default=6 Current=6 (Enter to keep Current):
Set value for timezone_offset. Default=0 Current=0 (Enter to keep Current):
Set value for established_days_age. Default=730 Current=730 (Enter to keep Current):
Set value for mode. Default=rdap Current=rdap (Enter to keep Current):
Set value for rdap_error_ttl_days. Default=7 Current=7 (Enter to keep Current):
Set value for freq_table. Default=freqtable2018.freq Current=freqtable2018.freq (Enter to keep Current):
Set value for enable_freq_scores. Default=True Current=True (Enter to keep Current):
Set value for freq_avg_alert. Default=5.0 Current=5.0 (Enter to keep Current):
Set value for freq_word_alert. Default=4.0 Current=4.0 (Enter to keep Current):
Set value for log_detail. Default=0 Current=0 (Enter to keep Current):
Set value for count_rdap_errors. Default=False Current=False (Enter to keep Current):
Set value for cache_browse_limit. Default=100 Current=100 (Enter to keep Current):
Commit Changes to disk?y
Configuration Written.
If this is a new installation you might consider preloading the top1m.import file with domain-stats-utils after starting the server.
Try:  domain-stats-utils -i /home/ubuntu/domain_stats/venv/lib/python3.12/site-packages/domain_stats/data/top1m.import -nx /opt/domain-stats-data
(venv) root@wazuhserver:/home/ubuntu/domain_stats/domain_stats/domain_stats#
```

Step 4: Start DNS-Stats Service

DNS-Stats is started using Gunicorn.

Purpose:
Expose DNS-Stats as a local HTTP API that Wazuh can query.

```
(venv) root@wazuhserver:/home/ubuntu/domain_stats/domain_stats/domain_stats# domain-stats /opt/domain-stats-data
Existing config found in directory. Using it.
Running /home/ubuntu/domain_stats/venv/bin/python3 -m gunicorn.app.wsgiapp 'domain_stats.server:config_app("/opt/domain-stats-data")' -c /opt/domain-stats-data/gunicorn_conf
ig.py
[2026-02-04 21:42:22 +0530] [34344] [INFO] Starting gunicorn 25.0.1
[2026-02-04 21:42:22 +0530] [34344] [INFO] Listening at: http://127.0.0.1:5730 (34344)
[2026-02-04 21:42:22 +0530] [34344] [INFO] Using worker: gthread
[2026-02-04 21:42:22 +0530] [34345] [INFO] Booting worker with pid: 34345
[2026-02-04 21:42:22 +0530] [34346] [INFO] Booting worker with pid: 34346
[2026-02-04 21:42:22 +0530] [34347] [INFO] Booting worker with pid: 34347
[2026-02-04 21:42:22 +0530] [34348] [INFO] Booting worker with pid: 34348
[2026-02-04 21:42:23 +0530] [34349] [INFO] Booting worker with pid: 34349
Using config /opt/domain-stats-data/domain_stats.yaml
Using config /opt/domain-stats-data/domain_stats.yaml
Using config /opt/domain-stats-data/domain_stats.yaml
Using config /opt/domain-stats-data/domain_stats.yaml
Using config /opt/domain-stats-data/domain_stats.yaml
```

Step 6: Validate Listening Port

The listening port is verified using system networking tools.

Purpose:
validation that DNS-Stats is reachable before SIEM integration.

```
root@wazuhserver:/home/ubuntu# netstat -ltpnd
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:443             0.0.0.0:*               LISTEN      591/node
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      1/init
tcp        0      0 127.0.0.54:53           0.0.0.0:*               LISTEN      338/systemd-resolve
tcp        0      0 0.0.0.0:1515            0.0.0.0:*               LISTEN      26547/wazuh-authd
tcp        0      0 0.0.0.0:1514            0.0.0.0:*               LISTEN      26634/wazuh-remoted
tcp        0      0 0.0.0.0:55000           0.0.0.0:*               LISTEN      26494/python3
tcp        0      0 127.0.0.1:5730          0.0.0.0:*               LISTEN      34344/python3
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      338/systemd-resolve
tcp6       0      0 :::22                   :::*                    LISTEN      1/init
tcp6       0      0 :::80                   :::*                    LISTEN      710/apache2
tcp6       0      0 :::9200                 :::*                    LISTEN      592/java
tcp6       0      0 :::9300                 :::*                    LISTEN      592/java
tcp6       0      0 :::55000                :::*                    LISTEN      26494/python3
```

Step 5: Test DNS-Stats API

From another terminal, DNS-Stats is tested using curl:

curl http://127.0.0.1:5730/google.com

Purpose:
Confirm the service is running and returning valid JSON responses.

```
root@wazuhserver:/home/ubuntu# curl http://127.0.0.1:5730/google.com
{"alerts":["YOUR-FIRST-CONTACT"],"category":"ESTABLISHED","freq_score":[6.6009,5.0887],"seen_by_isc":"RDAP","seen_by_web":"Mon, 15 Sep 1997 04:00:00 GMT","seen_by_you":"Wed,
 04 Feb 2026 16:14:23 GMT"}
root@wazuhserver:/home/ubuntu#
```

Step 7: Create Custom DNS-Stats Integration Script (Bash)

A custom **bash wrapper** is created for Wazuh.

Purpose:
Allows Wazuh to execute DNS-Stats queries when DNS events are detected.

```
  GNU nano 7.2                                                    custom-dnsstats *
#!/bin/sh
WPYTHON_BIN="framework/python/bin/python3"

SCRIPT_PATH_NAME="$0"

DIR_NAME="$(cd $(dirname ${SCRIPT_PATH_NAME}); pwd -P)"
SCRIPT_NAME="$(basename ${SCRIPT_PATH_NAME})"

case ${DIR_NAME} in
    */active-response/bin | */wodles*)
        if [ -z "${WAZUH_PATH}" ]; then
            WAZUH_PATH="$(cd ${DIR_NAME}/../..; pwd)"
        fi

        PYTHON_SCRIPT="${DIR_NAME}/${SCRIPT_NAME}.py"
    ;;
    */bin)
        if [ -z "${WAZUH_PATH}" ]; then
            WAZUH_PATH="$(cd ${DIR_NAME}/..; pwd)"
        fi

        PYTHON_SCRIPT="${WAZUH_PATH}/framework/scripts/${SCRIPT_NAME}.py"
```

Step 8: Create Custom DNS-Stats Python Script

A Python script is created that:

- Extracts the queried domain from the event
- Sends it to DNS-Stats
- Returns structured JSON to Wazuh

Purpose:
This is the **core integration logic** between Wazuh and DNS-Stats.

```
  GNU nano 7.2                                                custom-dnsstats.py *
#!/usr/bin/env python
# Aurora Networks Managed Services
# https://www.auroranetworks.net
# info@auroranetworks.net.
#
# This program is free software; you can redistribute it
# and/or modify it under the terms of the GNU General Public
# License (version 2) as published by the FSF - Free Software
# Foundation.
import sys
import os
from socket import socket, AF_UNIX, SOCK_DGRAM
from datetime import date, datetime, timedelta
import time
import requests
from requests.exceptions import ConnectionError
import json
pwd = os.path.dirname(os.path.dirname(os.path.realpath(__file__)))
socket_addr = '{0}/queue/sockets/queue'.format(pwd)
def send_event(msg, agent = None):
    if not agent or agent["id"] == "000":
        string = '1:dns_stats:{0}'.format(json.dumps(msg))
```

Step 9: Set Permissions on Scripts

Execution permissions are applied to all custom scripts.

Purpose:
Ensure Wazuh can execute the scripts securely.

```
root@wazuhserver:/var/ossec/integrations# chown root:wazuh custom-dnsstats custom-dnsstats.py
root@wazuhserver:/var/ossec/integrations# chown root:wazuh custom-dnsstats custom-dnsstats
root@wazuhserver:/var/ossec/integrations# chmod 750 custom-dnsstats custom-dnsstats.py
root@wazuhserver:/var/ossec/integrations# chmod 750 custom-dnsstats custom-dnsstats
root@wazuhserver:/var/ossec/integrations# ls -la | grep dns
-rwxr-x---  1 root wazuh   844 Feb  4 21:53 custom-dnsstats
-rwxr-x---  1 root wazuh  2772 Feb  4 21:53 custom-dnsstats.py
```

tep 10: Add DNS-Stats Integration to Wazuh

An <integration> block is added to Wazuh manager configuration.

Purpose:
Tell Wazuh **when** and **how** to call DNS-Stats.

```
<integration>
  <name>custom-dnsstats</name>
  <rule_id>101100</rule_id>
  <alert_format>json</alert_format>
</integration>
```

Step 11: Add DNS-Stats Detection Rules

Custom Wazuh rules are added for:

- First-seen domains
- Low-frequency domains
- Suspicious DNS patterns

Purpose:
Convert DNS-Stats output into meaningful alerts.

```
  GNU nano 7.2                                          /var/ossec/etc/rules/dns_stats.xml *
<group name="dnsstat,">
 <rule id="100010" level="5">
    <field name="integration">dnsstat</field>
    <description>DNS Stats</description>
    <options>no_full_log</options>
    <group>dnsstat_alert,</group>
  </rule>
<rule id="100011" level="5">
    <if_sid>100010</if_sid>
    <field name="dnsstat.alerts">LOW-FREQ-SCORES|SUSPECT-FREQ-SCORE</field>
    <description>DNS Stats - Low Frequency Score in Queried Domain</description>
    <mitre>
     <id>T1071</id>
    </mitre>
    <options>no_full_log</options>
    <group>dnsstat_alert,</group>
  </rule>
```

Step 12: Configure AlienVault OTX Integration Script

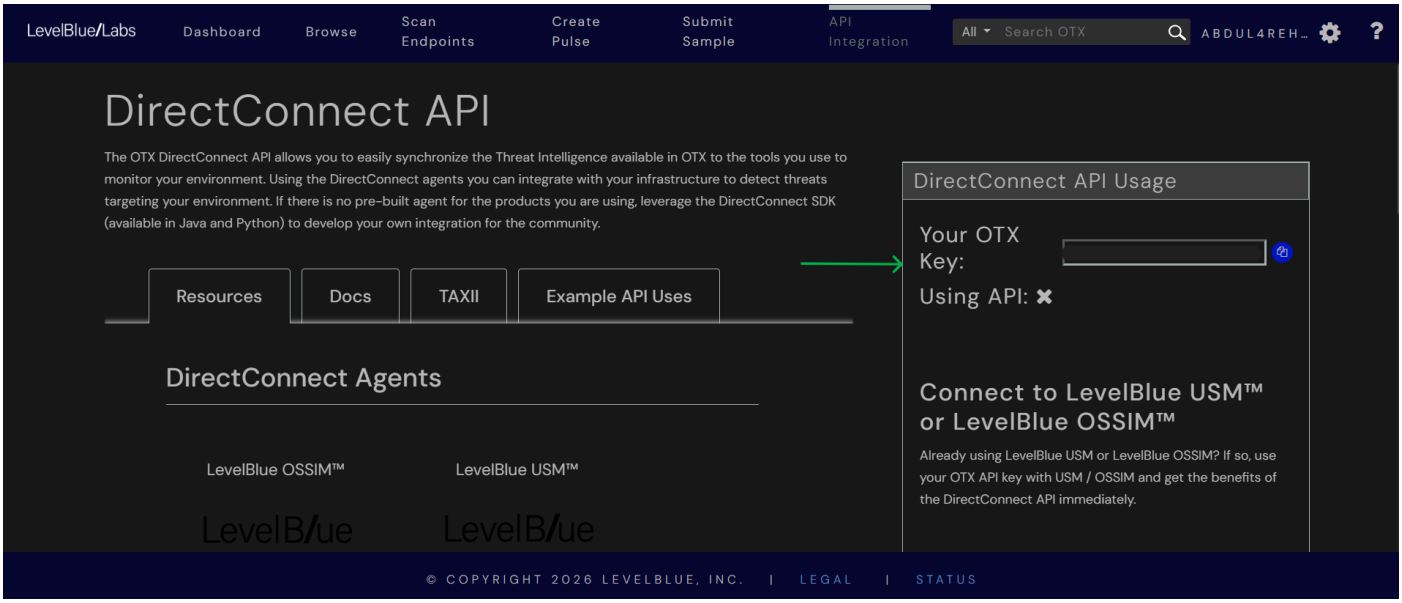
A Python script is created with:

- AlienVault OTX API key
- Domain lookup logic
- IOC parsing

Purpose:
Enrich DNS alerts with real-world threat intelligence.

```
  GNU nano 7.2                                    /var/ossec/integrations/custom-alienvault.py *
#!/var/ossec/framework/python/bin/python3
## Alienvault API Integration
#
import sys
import os
from datetime import date, datetime, timedelta
import time
import requests
from requests.exceptions import ConnectionError
import json
import ipaddress
import hashlib
import re
from socket import socket, AF_UNIX, SOCK_DGRAM
pwd = os.path.dirname(os.path.dirname(os.path.realpath(__file__)))
socket_addr = '{0}/queue/sockets/queue'.format(pwd)

def send_event(msg, agent = None):
    message = f"1:alienvault:{msg}"
    sock = socket(AF_UNIX, SOCK_DGRAM)
    sock.connect(socket_addr)
    sock.send(message.encode())
```



Step 13: Set Permissions on Script

Execution permissions are applied to the custom-alienvault custom script.

Purpose:
Ensure Wazuh can execute the scripts securely.

```
root@wazuhserver:/var/ossec/integrations# chown root:wazuh /var/ossec/integrations/custom-alienvault.py
chmod 750 /var/ossec/integrations/custom-alienvault.py
```

Step 14: Add AlienVault OTX Rules

Rules are added to detect:

- Malicious indicators returned by AlienVault OTX
- IOC matches

Purpose:
Map threat intelligence findings to alert severity.

```
  GNU nano 7.2                                  /var/ossec/etc/rules/alienvault_otx_rules.xml *
<group name="alienvault,">
<rule id="91580" level="12">
  <decoded_as>json</decoded_as>
  <field name="sections">\.+</field>
  <field name="type">\.+</field>
  <description>AlienVault OTX -Indicator(s) Found</description>
  <mitre>
    <id>T1036</id>
  </mitre>
  <options>no_full_log</options>
  <group>otx_ioc,</group>
</rule>
</group>
```

Step 15: Add AlienVault Integration to Wazuh

Another <integration> block is added for AlienVault.

Purpose:
Chain DNS detection → DNS-Stats → AlienVault enrichment.

```
<integration>
  <name>custom-alienvault</name>
  <group>dnsstat_alert</group>
  <alert_format>json</alert_format>
</integration>
```

Step 16: Create PowerShell Script on Windows Endpoint

A PowerShell script is created on the Windows endpoint.

Purpose:
Used during testing and as part of Active Response validation.

otx.ps1

File Edit View

```
#################################
### Script to check event data on AlienVault OTX IoCs
### Aurora Networks Managed Services
### https://www.auroranetworks.net
### info@auroranetworks.net
#################################
##########
# The API Call to OTX will run the parameter passed in the call against existing IoCs
# The API response is filtered out to only get IoCs part of pulses created by the user "AlienVault"
# API Response (relevant fields) in the response converted to JSON and appended to active-responses.log
# An API key to access AlienVault OTX is required (otx.alienvault.com)
##########
```

Step 17: Create Active Response CMD Script

A .cmd wrapper is placed in:

ossec-agent/active-response/bin

Purpose:
Allows Wazuh to trigger AlienVault checks from the endpoint if required.

Simple script to run AlienVault OT

File Edit View H1

```
:: Simple script to run AlienVault OTX PShell script.
:: The script executes a powershell script and appends output.
@ECHO OFF
ECHO.

"C:\Program Files\PowerShell\7\"pwsh.exe -executionpolicy ByPass -File "C:\Users\Abdul Rehman\Downloads\otx.ps1"

:Exit
```

Step 18: Configure Active Response in Wazuh

Active response block added to Wazuh configuration so to trigger it.

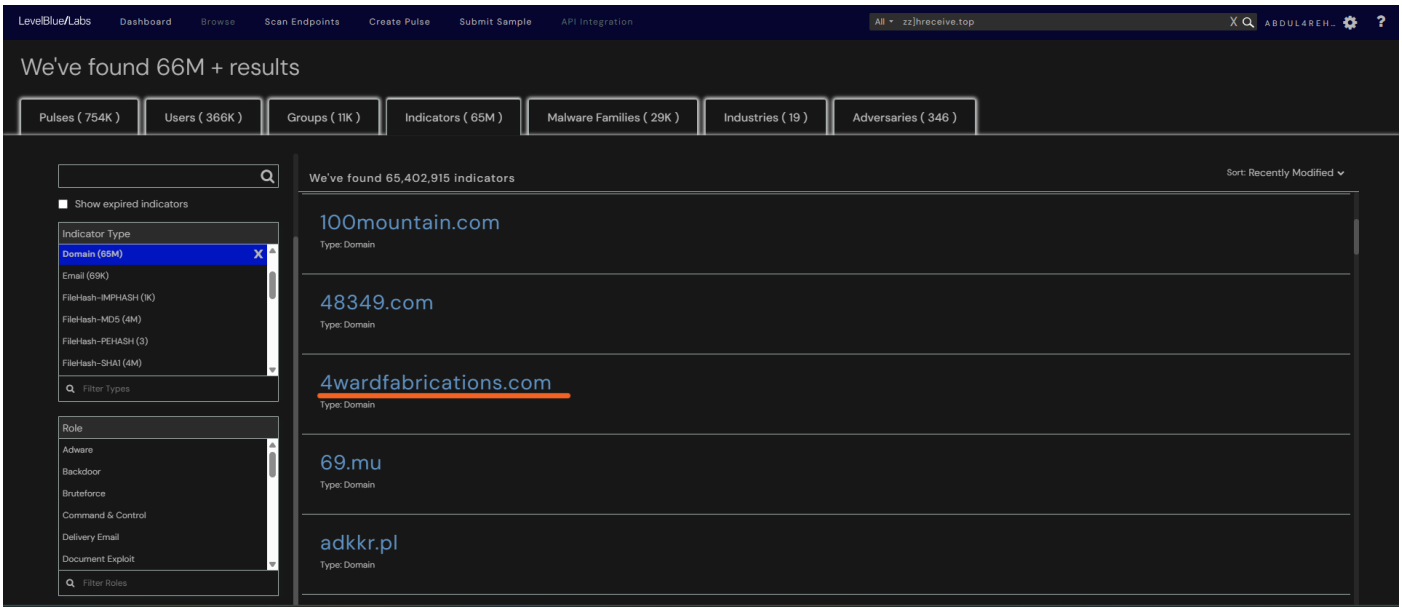
Purpose:
Enable automated responses or enrichment actions.

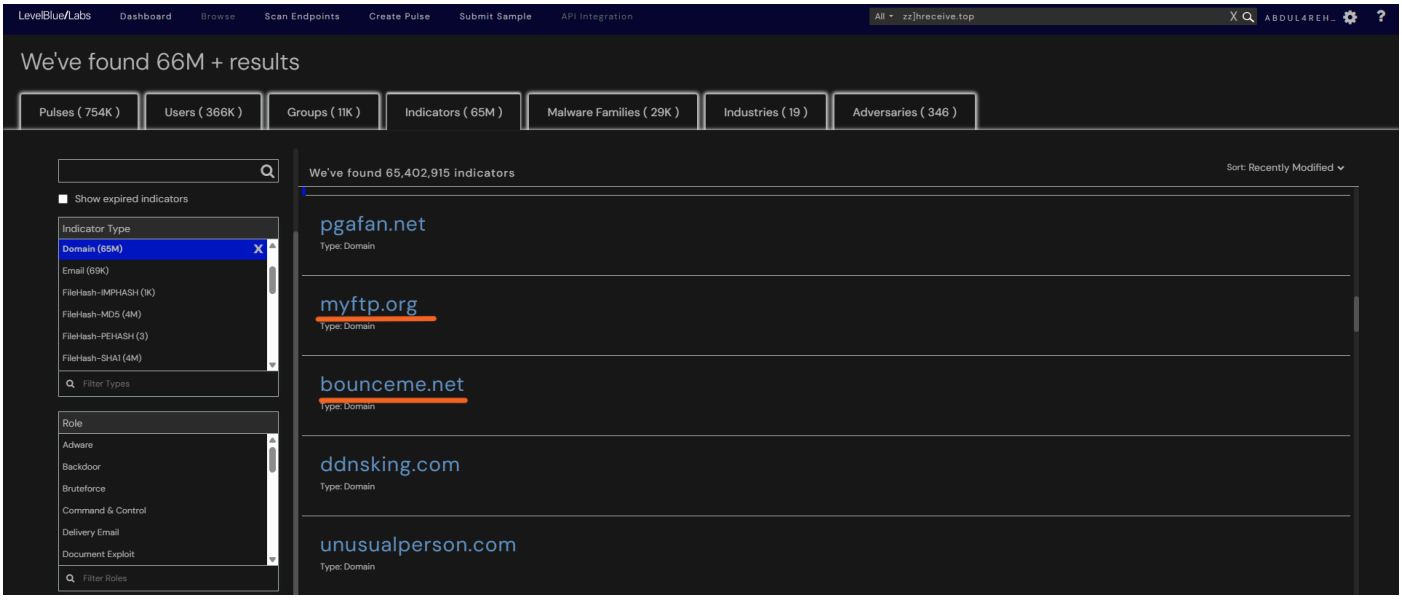
```
<command>
  <name>alienvault_otx</name>
  <executable>otx.cmd</executable>
  <timeout_allowed>no</timeout_allowed>
</command>
<active-response>
  <disabled>no</disabled>
  <level>3</level>
  <command>alienvault_otx</command>
  <location>local</location>
  <rules_group>dnsstat_alert</rules_group>
</active-response>
```

Step 19: Identify Malicious Domains in AlienVault OTX

Known malicious domains are selected from OTX.

Purpose:
Provide realistic test data.





Step 20: Generate DNS Queries on Windows

Using PowerShell:

Resolve-DnsName <domain>

Purpose:
Trigger Sysmon Event ID 22.

```
PS C:\Users\Abdul Rehman> Resolve-DnsName 4wardfabrications.com

Name                                           Type   TTL   Section    IPAddress
----                                           ----   ---   -------    ---------
4wardfabrications.com                          A      3600  Answer     169.47.130.72

Name       : 4wardfabrications.com
QueryType  : NS
TTL        : 3600
Section    : Authority
NameHost   : ns2.afraid.org
```

```
PS C:\Users\Abdul Rehman> Resolve-DnsName myftp.org

Name                                           Type   TTL   Section    IPAddress
----                                           ----   ---   -------    ---------
myftp.org                                      A      60    Answer     158.247.7.206


PS C:\Users\Abdul Rehman> Resolve-DnsName bounceme.net

Name                                           Type   TTL   Section    IPAddress
----                                           ----   ---   -------    ---------
bounceme.net                                   A      60    Answer     158.247.7.206
```

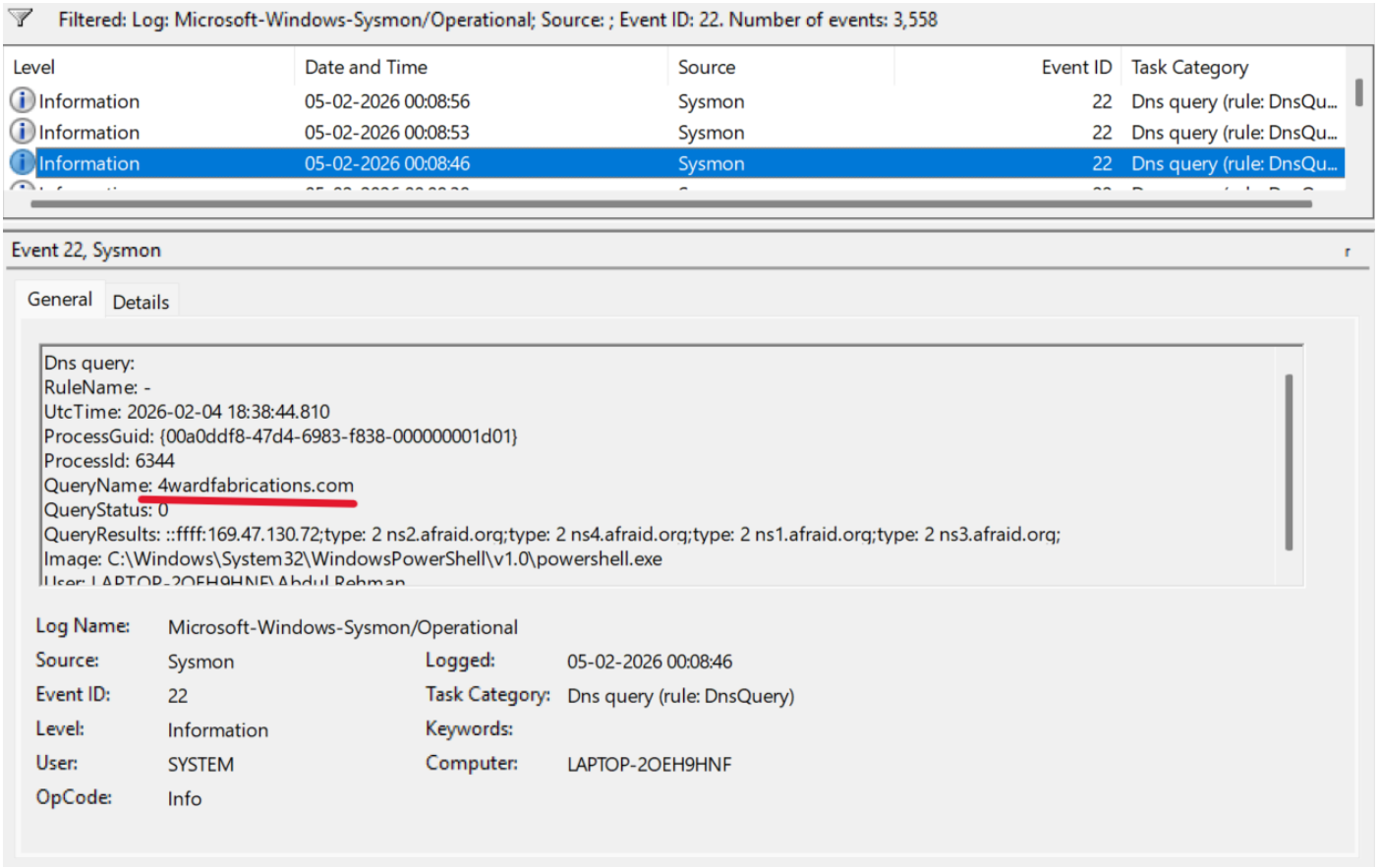
Step 21: Verify Sysmon DNS Events

Sysmon logs confirm:

- Event ID 22
- Queried domain

- Process name

Purpose:
Ensure endpoint telemetry is correct.

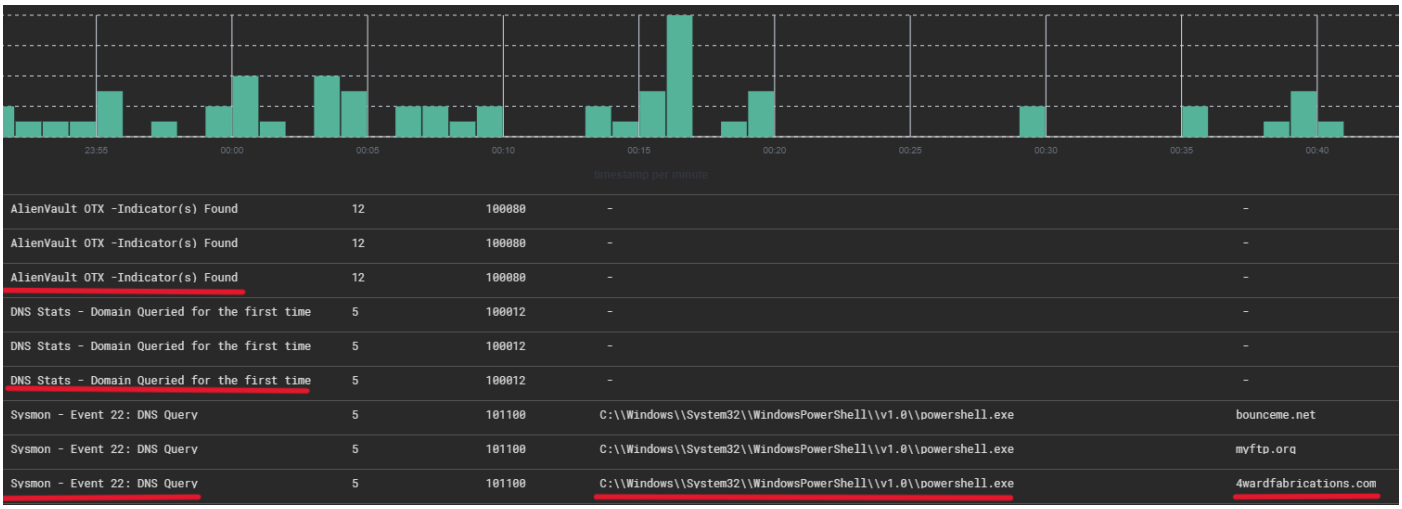


Step 22: Observe Alert Chain in Wazuh

Alerts appear in this order:

1. DNS Query detected
2. DNS-Stats analysis
3. AlienVault IOC match

Purpose:
Validate end-to-end correlation.



Step 23: Analyze Individual Alerts

Detailed alert view shows:

- Domain queried
- Frequency score
- First seen timestamp

t _index	wazuh-alerts-4.x-2026.02.04
t agent.id	008
t agent.ip	[redacted]
t agent.name	windows2
t data.win.eventdata.image	C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe
t data.win.eventdata.processGuid	{00a0ddf8-47d4-6983-f838-000000001d01}
t data.win.eventdata.processId	6344
t data.win.eventdata.queryName	4wardfabrications.com
t data.win.eventdata.queryResults	::ffff:169.47.130.72;type: 2 ns2.afraid.org;type: 2 ns4.afraid.org;type: 2 ns1.afraid.org;type: 2 ns3.afraid.org;
t data.win.eventdata.queryStatus	0
t data.win.eventdata.user	LAPTOP-2OEH9HNF\\Abdul Rehman
t data.win.eventdata.utcTime	2026-02-04 18:38:44.810
t data.win.system.channel	Microsoft-Windows-Sysmon/Operational
t data.win.system.computer	LAPTOP-2OEH9HNF
t data.win.system.eventID	22
t data.win.system.eventRecordID	297391
t data.win.system.keywords	0x8000000000000000

DNS-Stats Enrichment Analysis

Analyst can see:

- Domain rarity
- Historical presence
- RDAP/WHOIS data
- MITRE ATT&CK mapping

t	_index	wazuh-alerts-4.x-2026.02.04
t	agent.id	008
t	agent.ip	[redacted]
t	agent.name	windows2
t	data.dnsstat.alerts	YOUR-FIRST-CONTACT
t	data.dnsstat.category	ESTABLISHED
#	data.dnsstat.freq_score	7.264, 7.026
t	data.dnsstat.query	4wardfabrications.com
t	data.dnsstat.seen_by_isc	RDAP
t	data.dnsstat.seen_by_web	Sun, 22 Feb 2015 21:58:22 GMT
t	data.dnsstat.seen_by_you	Wed, 04 Feb 2026 18:44:17 GMT
t	data.integration	dnsstat
t	decoder.name	json
t	id	1770230658.51357015
t	input.type	log
t	location	dns_stats
t	manager.name	wazuhserver
t	rule.description	DNS Stats - Domain Queried for the first time
t	rule.groups	dnsstat, dnsstat_alert
t	rule.id	100012
#	rule.level	5
◪	rule.mail	false
t	rule.mitre.id	T1071
t	rule.mitre.tactic	Command and Control
t	rule.mitre.technique	Application Layer Protocol
🗓	timestamp	Feb 5, 2026 @ 00:14:18.071

AlienVault IOC Analysis

Final alert confirms:

- Malicious indicator
- Threat category
- MITRE tactics and techniques

t	_index	wazuh-alerts-4.x-2026.02.04
t	agent.id	000
t	agent.name	wazuhserver
t	data.base_indicator.access_type	public
t	data.base_indicator.id	2117777819
t	data.base_indicator.indicator	4wardfabrications.com
t	data.base_indicator.type	domain
t	data.dns_requested	4wardfabrications.com
t	data.message	Alienvault OTX API response successfully parsed
t	data.sections	general, geo, url_list, passive_dns, malware, whois, http_scans
t	data.status	success
t	data.type	domain
t	decoder.name	json
t	id	1770230797.51590027
t	input.type	log
t	location	alienvault
t	manager.name	wazuhserver
t	rule.description	AlienVault OTX -Indicator(s) Found
t	rule.groups	alienvault, otx_ioc
t	rule.id	100080
#	rule.level	12
◪	rule.mail	true
t	rule.mitre.id	T1036
t	rule.mitre.tactic	Defense Evasion
t	rule.mitre.technique	Masquerading

Analyst Value & SOC Benefits

-
- Detects **early-stage infections**
- Identifies **new or rare domains**
- Reduces false positives
- Adds **threat context automatically**
- Enables **proactive threat hunting**
- Improves **MTTD and MTTR**

Skills & Concepts Gained

- SIEM integration engineering
- Threat intelligence enrichment
- DNS-based threat hunting
- Wazuh custom integrations
- Sysmon telemetry analysis
- MITRE ATT&CK mapping
- Active Response workflows
- SOC alert tuning

Conclusion

This project demonstrates a **real-world SOC detection pipeline** where raw DNS telemetry is transformed into **actionable intelligence**.

By integrating DNS-Stats and AlienVault OTX with Wazuh:

- Analysts gain context
- Alerts gain meaning
- Threats are detected earlier

This setup closely mirrors **enterprise SOC architectures** used in production environments.
